RAG Pipeline

Ingestion Pipeline:

1. User submits a public GitHub URL through the frontend
2. Check the repos collection for an existing entry
    a. If an entry does exist, compare the existing commit SHA with the most recent one from the repo
        i. If it matches, just update the repo's lastAccessed and end the pipeline
        ii. If it's different, continue with the pipeline
    b. If an entry doesn't exist, just continue along with the pipeline
3. Using the user-given URL use simple-git to clone the repo to a specified location
4. Loop through all the files using the ignore directory and the valid files list. Use the directory list to ignore scanning unnecessary files, and use the valid files list to see if a file is supported and accepted by the next steps
5. Generate a prediction of how much space this repo will take up using the size of all valid files and a predetermined size for each chunk. A buffer is also added to account for prediction inaccuracy
6. Using the predicted size, check if the size goes above the set limit for each repo
    a. If it's too large, reject the repo ingestion, clean up cloned files, and give the user a valid reason
    b. If it's under the limit, continue with the pipeline
7. Check if the database has enough space to accept this repo ingestion (note: the database limit will be lowered to 80-95% of its actual limit, to leave a cushion for other database activities)
    a. If it doesn't have enough, delete repos in a loop until the database has enough room for the new repo. These repos will be deleted based on the lastAccessed date, with the older ones being deleted first
    b. If it does have enough space, continue with the pipeline
8. Now, check one more time if the repo already exists
    a. If yes, update the repo in the database with the new lastAccessed date and new SHA
    b. If no, initialize a new repo in the database with the URL and latest SHA
9. Create a new repo file tree and update the repo in the database
10. Clean up all the cloned repo files by deleting them
11. Create a concrete syntax tree using Tree Sitter
12. Loop through all the nodes in the generated tree, and for each chunk, do the following
    a. Create an embedding of the code block
    b. Generate metadata for the code block, which consists of the repo URL, the relative path of the file from the root, the file name where the code block is from, the name of the code block, and the type of the code block, like class or function, the language, the parent directory of the file, and the start and end lines
    c. Bundle the code block, embedding, and metadata together
    d. Save to database

Query Pipeline:

1. User submits a query along with an already ingested repo
2. Semantic cache check
3. Check if the question is relevant to the website and sanitize the query
    a. If it's not relevant, reject the request along with an apology message
4. Update the query being used for the rest of the pipeline to the sanitized query
5. Update the lastAccessed for the repo being queried
6. Using the sanitized query, run the interpreter. This will generate a hypothetical chunk that will be the ideal code block for the query, a possible language, and a directory if specified in the sanitized query, and an embedding based on the hypothetical chunk
7. If the embedding for the hypothetical chunk fails
    a. Reject the query and send an apology message
8. Perform a vector search using the embedded hypothetical chunk
9. After receiving the n number of chunks for the search, apply the following post-retrieval filters
    a. Score threshold - this is the most important filter for relevance. Using a predetermined number, chunks are filtered out using the vector search similarity score
    b. Directory filter - if the query specified a directory and that directory was isolated by the interpreter, filter the chunks whose parent directory metadata field includes that string (case-insensitive, fuzzy match)
    c. Overlap deduplication - this will remove chunks that have too much overlap in the same file. The one with the higher score will be kept
    d. Noise filter - filters out certain files that can add unnecessary noise to most questions, like index/entrypoint files, md files, event, frontend files (most questions will be about backend), logging, constants, and so on. These files are

filtered out based on name convention, and some of them have bypasses that will save them from being ignored. The list of bypass keywords is hard-coded. For instance, if a question includes bypass keywords for the frontend, like the question has words like client, frontend, react, it will bypass the noise filter and allow those chunks to pass through, but not all noise patterns have bypasses

   e. Per-file cap - this filter caps the number of chunks that can come from one file so as not to dominate the chunk pool. But this has a couple of bypasses and changes, too. If the chunks are all from the same file and the score confidence is high, the file cap will be skipped. Also, if the chunks have high file diversity, the cap is lowered to allow more chunks from different files to broaden the context

10. If the chunks received after the filters are 1, re-run the filters with lower score thresholds
11. Re-rank the chunks. This step uses an LLM call to re-score all chunks based on the sanitized query, and then the chunks are re-ordered, and only the top n are taken. This ensures chunks are re-ranked again based on context and not just similarity, and also, only a strict number of chunks can make it through
12. If the context for the query is too large, perform a context compression. A context compression will be performed by a less costly LLM, and it will be told to extract only the parts of the code block that are relevant to the query, and leave everything else out
13. Build a query for the final LLM call using the original message and given context chunks
14. Send the generated system prompt and user message to the final LLM
15. Add to semantic cache
16. Send the generated response to the user along with stats about each chunk